

Cloud Native Architecture for Modern Platforms

21CSE477T

Dr. J. Justina Michael

Assistant Professor,
Department of Networking and Communications,
School of Computing, SRMIST - Kattankulathur.

Cloud Native Architecture for Modern Platforms

Unit 3: Micro Services

CLR3: Understand the core principles of software design

CO3: Design and Design and implement a scalable polyglot persistence layer including regular star schema and document store.

Theory - Key topics:

Micro services Architecture: Introduction Concept: Domain Drive Design, Modelling: Bounded Context, Workshop using end to end layering of components and cloud hosting, and Technology: Element of micro services and implementation mechanism, Technology: Container - Kubernetes part 2, Technology: Inter-micro services communication, Workshop for moving from monolithic to microservices components. CA Project milestone Archetype Submission, Serverless Architecture, Introduction Concept: Function as a service, Modelling: Functions, Technology: Elements of functions and workflow implementation mechanism

Unit 3

Micro services

Micro services Architecture: Introduction Concept

Introduction

- **Microservices Architecture (MSA)** is a **software architectural style** where an application is structured as a collection of small, **loosely coupled, independently deployable services**.
- Each service is designed to perform a **single business function** and communicates with other services using **lightweight protocols (typically HTTP/REST or gRPC, sometimes messaging queues like Kafka/RabbitMQ)**.
- It contrasts with the **monolithic architecture**, where all modules and components are tightly integrated into one large application.

Key Concepts

- **Independence** – Each microservice can be developed, deployed, and scaled independently.
- **Single Responsibility Principle** – A service is designed around one business capability (e.g., *Payment Service, Inventory Service, User Authentication Service*).
- **Technology Agnostic** – Different microservices can be built using different programming languages or databases as long as they follow agreed communication protocols (polyglot persistence).
- **Decentralized Data Management** – Each service manages its own database/schema to avoid tight coupling.
- **Communication** – Services interact using APIs, event-driven messaging, or service meshes.
- **Resilience & Fault Tolerance** – Failure in one service should not crash the whole system.
- **Scalability** – Services can be scaled horizontally (individually) based on demand.

Advantages

- Faster development and deployment cycles.
- Easier to adopt **DevOps & CI/CD pipelines**.
- Independent scaling reduces infrastructure costs.
- Improved fault isolation.
- Flexibility in technology choices.
- Easier to maintain and update specific parts of the application.

Challenges

- Complexity in design and management.
- Requires advanced **orchestration tools** (e.g., Kubernetes, Docker Swarm).
- Data consistency can be challenging (eventual consistency).
- Monitoring, logging, and debugging become harder.
- Higher communication overhead compared to monoliths.

Real-Time Applications of Microservices

- **Netflix** – Uses microservices to deliver personalized content streaming at scale.
- **Amazon** – Each function (orders, payments, recommendations) runs as an independent service.
- **Uber** – Separate services for ride-matching, payments, notifications, etc.
- **Spotify** – Music recommendation, playlist management, and streaming as separate services.
- **Banking/Fintech Apps** – Payments, fraud detection, customer accounts as services.

Tools & Technologies for Microservices

- **Containerization:** Docker
- **Orchestration:** Kubernetes, Docker Swarm
- **Service Discovery:** Consul, Eureka
- **API Gateways:** Kong, Nginx, AWS API Gateway
- **Messaging Queues:** Kafka, RabbitMQ
- **Monitoring:** Prometheus, ELK Stack

Domain Drive Design

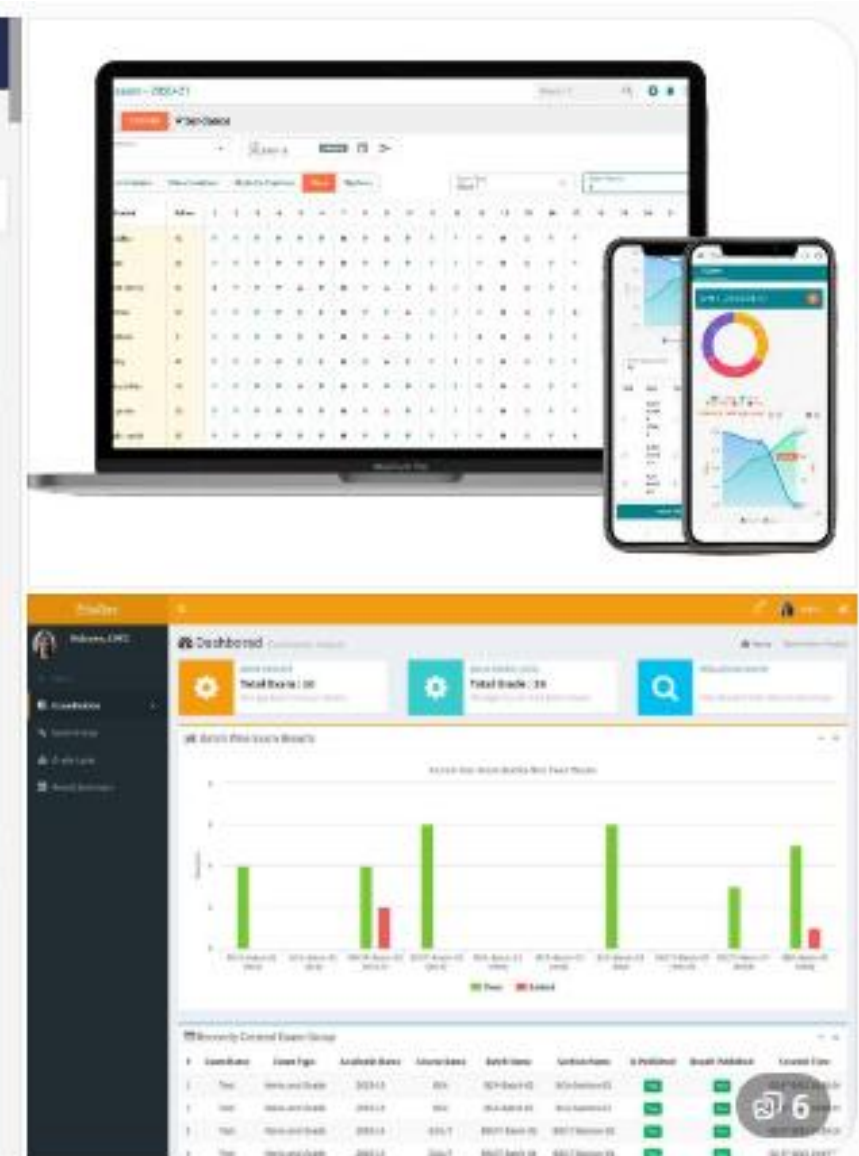
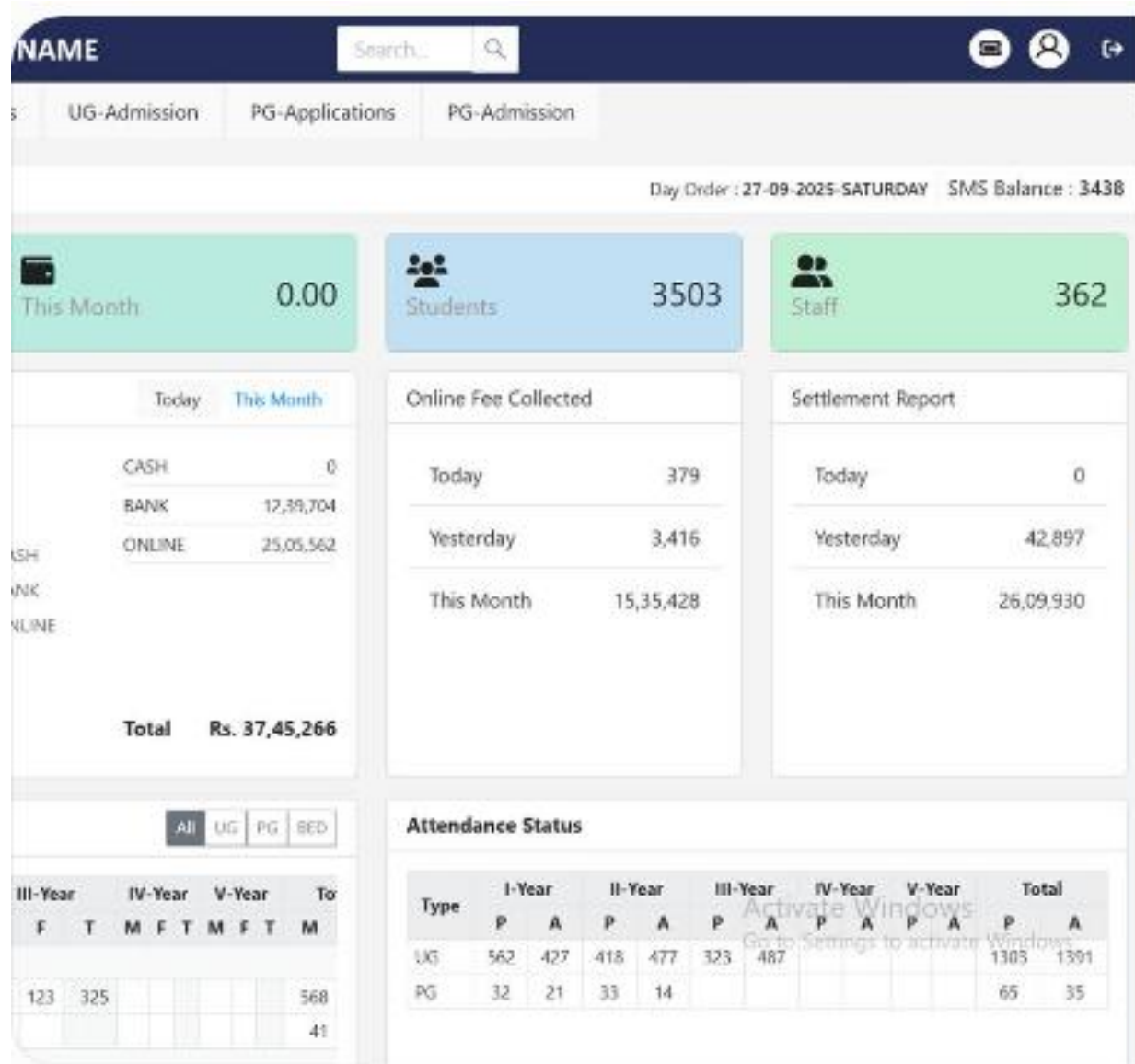
Introduction

- Domain-Driven Design (DDD) is a software design approach introduced by Eric Evans. It focuses on building software around the business domain instead of only the technical implementation.
- The idea: Instead of just focusing on technology, **software should be modeled around business problems and concepts.**
- It's especially relevant in **cloud-native and microservices environments**, where aligning services to business capabilities is crucial.

Domain Driven Design

Example:
College
ERP System

Monolithic
vs
MS, DDD



Core Concepts of DDD

- **Domain:** The sphere of knowledge, activity, or interest around which the application logic revolves (e.g., banking, healthcare, e-commerce).
- **Model:** An abstraction that describes selected aspects of the domain and helps solve problems.
- **Ubiquitous Language:** A common vocabulary used consistently by developers, domain experts, and stakeholders to reduce miscommunication.
- **Bounded Context:** A logical boundary where a particular domain model is defined and applicable.
 - Example: *In e-commerce, “Order Management” and “Inventory Management” are separate bounded contexts. Both use the word “Product”, but in different ways.*
- **Entities:** Objects that have an identity and persist over time (e.g., *Customer name*).
- **Value Objects:** Objects that are immutable and defined by their values; no identity (e.g., *Address*).
- **Aggregates:** A cluster of domain objects treated as a single unit (root entity + value objects).
- **Repositories:** Mechanisms for retrieving and storing domain objects.
- **Domain Events:** Events that capture things that happened in the domain (e.g., *OrderPlaced*, *PaymentProcessed*).

Advantages of DDD

- Aligns software closely with business requirements.
- Reduces miscommunication between developers & domain experts.
- Handles complexity by splitting the system into bounded contexts.
- Fits naturally with **microservices** (each bounded context → one microservice).
- Improves scalability and maintainability.

Challenges of DDD

- Requires deep domain knowledge.
- Needs strong collaboration between developers and business experts.
- Can introduce complexity in design.
- Not ideal for simple CRUD-based applications.

Real-Time Applications of DDD

- **Banking Systems** – Separate domains for Accounts, Loans, Transactions.
- **E-commerce** – Domains like Orders, Payments, Inventory, Shipping.
- **Healthcare** – Domains like Patient Records, Appointments, Billing.
- **Ride-Sharing Apps (Uber)** – Booking, Payment, Notifications as bounded contexts.
- **ERP Systems** – Modules aligned to business domains: HR, Finance, Supply Chain.

DDD in Cloud-Native and Microservices

- **Bounded Context = Microservice**
- Each microservice manages its own **domain logic & data**.
- Encourages **clear APIs, event-driven communication, and domain events**.
- Works well with cloud-native patterns: **CQRS (Command Query Responsibility Segregation), Event Sourcing, and API Gateways**.

Modelling: Bounded Context

Introduction

- **Bounded Context** is one of the **central patterns** in **Domain-Driven Design (DDD)**.
- A **bounded context** defines a **clear boundary** within which a particular domain model is **valid and consistent**.
- Inside this boundary:
 - Terms, concepts, and rules have a **specific meaning**.
 - Outside this boundary, the same term may have a **different meaning**.
- It prevents **ambiguity and model conflicts** when multiple teams work on large systems.

👉 Example:

In an **e-commerce platform**:

- “*Order*” in the **Order Management context** means a customer’s purchase.
- “*Order*” in the **Inventory context** might mean a supplier restock order.
- Same word, different meaning → hence **separate bounded contexts**.

Why Bounded Contexts are Important

- Large systems have multiple **sub-domains** (e.g., payments, shipping, inventory).
- Without clear boundaries, the model becomes inconsistent and complex.
- Bounded Context ensures **model clarity, team autonomy, and scalability**.
- In **cloud-native microservices**, bounded contexts map directly to **microservices boundaries**.

Core Principles

- **Explicit Boundaries** – Defines what's inside and outside the context.
- **Ubiquitous Language within Context** – Terms have one meaning within a context.
- **Context Mapping** – Shows how multiple bounded contexts relate to each other (shared kernel, customer-supplier, anti-corruption layer).
- **Autonomy** – Each context evolves independently.

Advantages

- Prevents domain model confusion.
- Simplifies communication between business experts and developers.
- Enables **team autonomy** (each team owns one bounded context/microservice).
- Aligns naturally with **microservices architecture**.
- Improves **maintainability and scalability**.

Challenges

- Requires deep analysis of business processes.
- Risk of **over-segmentation** if too many contexts are created.
- Inter-context communication can become complex.
- Needs good documentation and collaboration.

Real-Time Applications of Bounded Contexts

- **Amazon (E-commerce):** Contexts like *Customer Service, Order Management, Inventory, Payments, Shipping*.
- **Banking Systems:** *Accounts, Loans, Payments, Fraud Detection* each modeled as separate bounded contexts.
- **Healthcare Apps:** *Patient Records, Billing, Appointments, Pharmacy*.
- **Ride-Sharing (Uber):** *Rider Booking, Driver Management, Payment, Notifications* contexts.

Bounded Context in Cloud-Native Systems

- Each bounded context → one microservice.
- Services communicate via APIs or events.
- Example: *Order Management service publishes “OrderPlaced” event → Inventory service consumes it.*
- Helps implement scalability, fault isolation, and independent deployments.

Workshop using end to end layering of components and cloud hosting, and Technology

Common Layers in an End-to-End Cloud-Native Application

1. Presentation Layer (Client Layer) - UI & user interaction
2. Application Layer (Business Logic Layer) - Business rules
3. Data Layer (Persistence Layer) - Storage & retrieval
4. Integration Layer (Optional in Complex Apps) - External systems & APIs

Example - Food Delivery App - What happens?

- Presentation: “I want this pizza.”
- Application: “Let’s check it’s possible, calculate price, and confirm.”
- Data: “Save the order & get customer info.”
- Integration: “Charge the customer, inform the kitchen, send a receipt.”

Mini Application: *Student Feedback API*

👉 Covers UI → API → Database layering, but in a very simple way.

1. Frontend (HTML Form)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Student Feedback</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Feedback Form</h2>
```

```
  <form action="/feedback" method="POST">
```

```
    <input type="text" name="name" placeholder="Your Name" required><br>
```

```
    <textarea name="message" placeholder="Your Feedback" required></textarea><br>
```

```
    <button type="submit">Submit</button>
```

```
  </form>
```

```
</body>
```

```
</html>
```

2. Backend (Node.js + Express)

```
const express = require("express");  
const bodyParser = require("body-parser");  
const app = express();  
app.use(bodyParser.urlencoded({ extended: true }));  
let feedbacks = [];  
app.get("/", (req, res) => {  
  res.send("Welcome to Feedback App!");  
});  
app.post("/feedback", (req, res) => {  
  feedbacks.push({ name: req.body.name, message: req.body.message });  
  res.send("Thanks for your feedback!");  
});  
app.get("/all-feedback", (req, res) => {  
  res.json(feedbacks);  
});  
app.listen(3000, () => console.log("App running on http://localhost:3000));
```

//Imports the Express library
// Imports the body-parser middleware module
// Creates an Express application
// In-memory storage (no DB for simplicity)
// Defines a GET route for the root path /.
// Defines a POST route at /feedback
// Starts the HTTP server and listens for incoming requests on port 3000.

Element of micro services and implementation mechanism

Elements of Microservices Architecture

a) Service

- The smallest unit in microservices.
- Performs one task (e.g., Payment service).
- Runs independently and communicates with others via APIs.

b) API Gateway

- Entry point for all client requests.
- Routes requests to the correct service.
- Handles security, load balancing, and logging.
- Example: Netflix API Gateway.

c) Database per Service

- Each microservice can have its **own database** (ensures loose coupling).
- Example: User Service uses MongoDB, Payment Service uses MySQL.

d) Service Discovery

- Helps microservices **find each other** automatically.
- Example: Netflix Eureka, Consul.

e) Load Balancer

- Distributes requests evenly across instances of microservices.
- Ensures high availability.

f) Inter-service Communication

- Microservices communicate using:
 - Synchronous:** REST APIs / gRPC
 - Asynchronous:** Message brokers (Kafka, RabbitMQ).

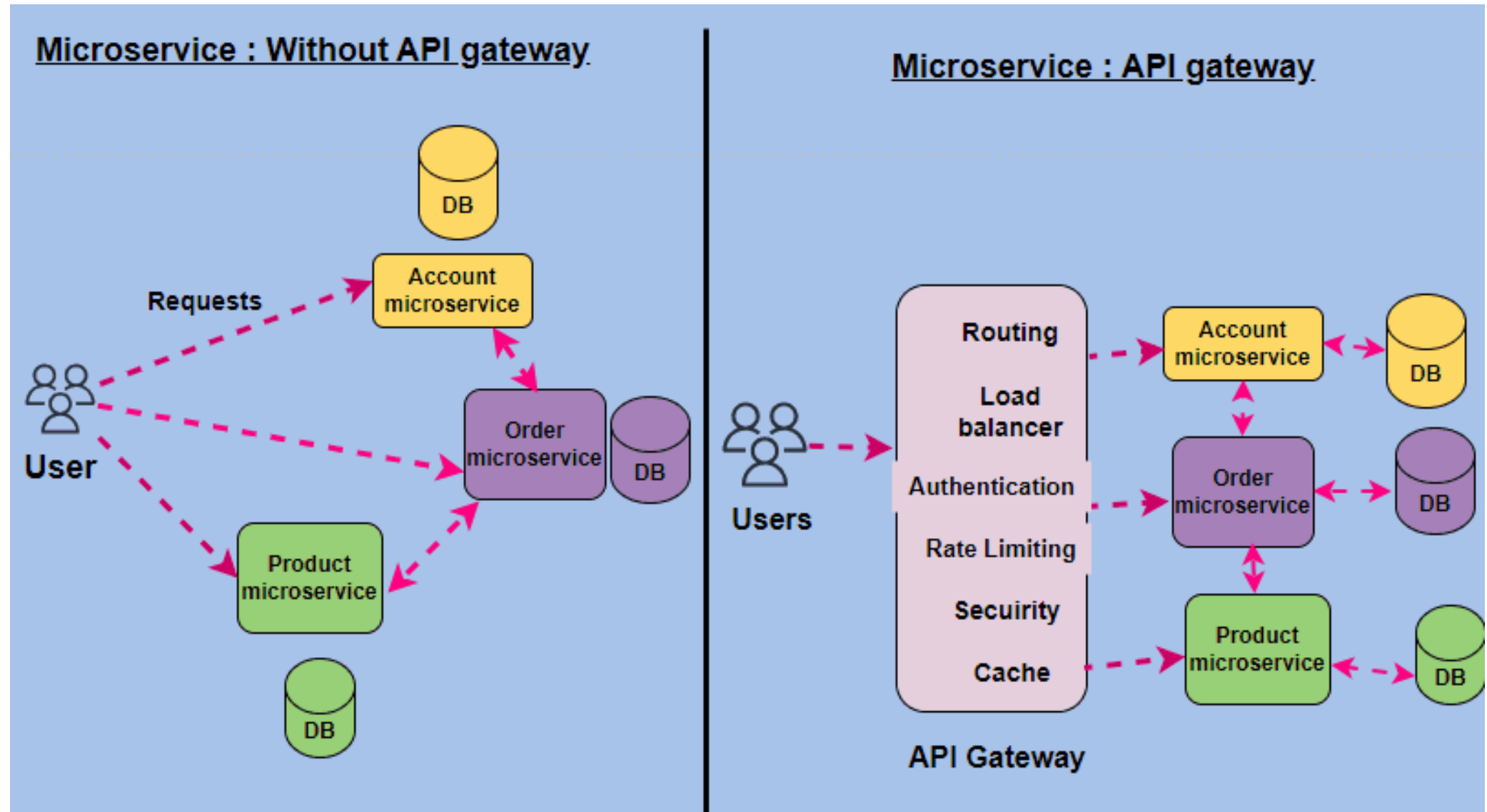
g) Security

- Authentication and authorization between services (OAuth2, JWT).

h) Monitoring and Logging

- Essential to track system health and performance.
- Tools: Prometheus, ELK Stack (Elasticsearch, Logstash, Kibana).

Element of micro services and implementation mechanism



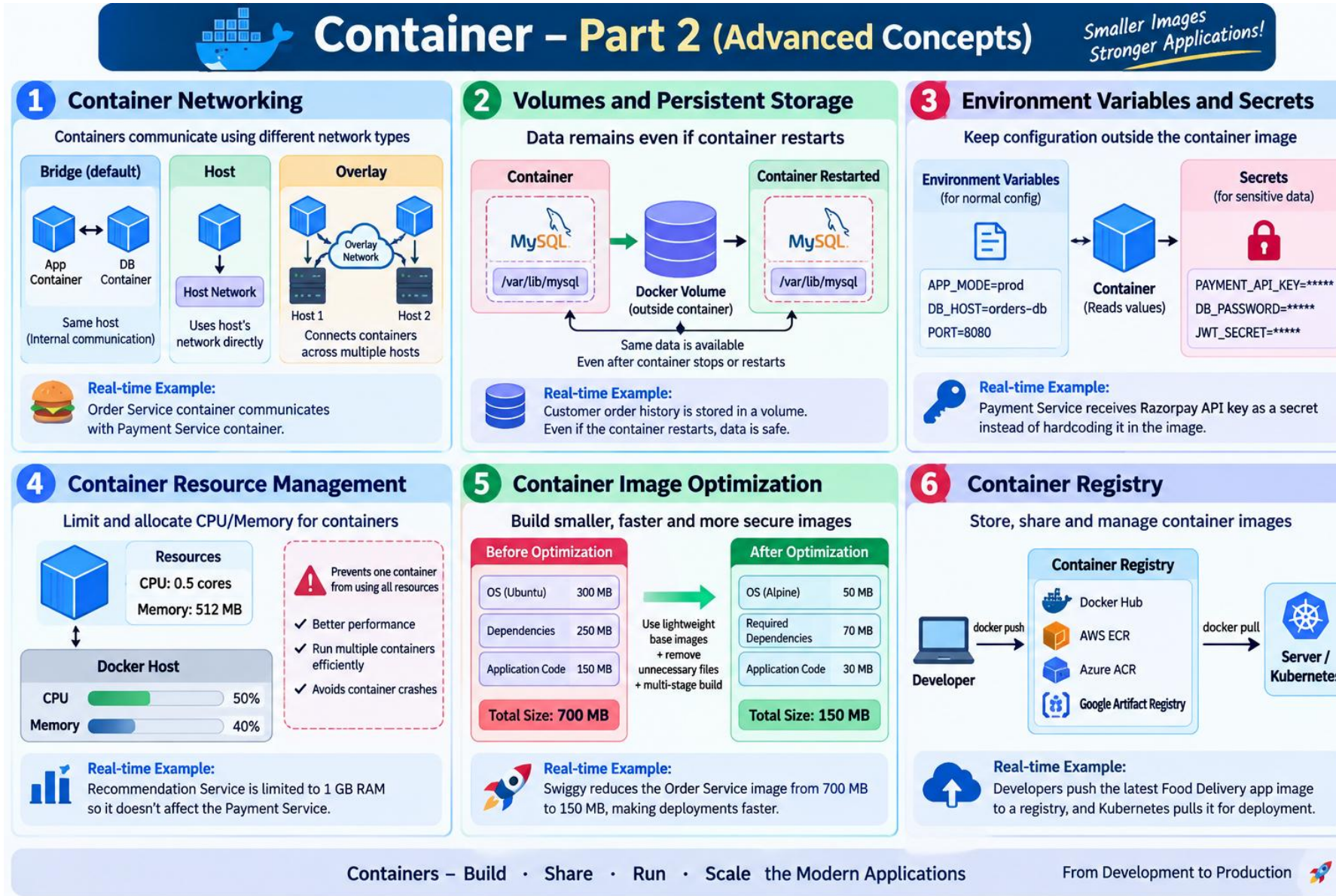
Implementation Mechanism

- **Decomposition**
 - Break down a monolith into small services.
 - Example: A banking app → Account Service, Loan Service, Transaction Service.
- **Containerization** - Package microservices in **containers** (Docker) for portability.
- **Orchestration** - Manage containers using Kubernetes (scaling, self-healing, rolling updates).
- **Communication** - Use REST, gRPC, or messaging queues (Kafka, RabbitMQ).
- **Data Management**
 - Each service has its own database schema.
 - Use **event sourcing** or **CQRS** (Command Query Responsibility Segregation) when needed.
- **Deployment**
 - CI/CD pipelines (Jenkins, GitHub Actions, GitLab CI).
 - Automated testing + rolling deployments.
- **Monitoring** - Use centralized logging (ELK) + monitoring (Prometheus + Grafana).

Real-Time Applications of Microservices

- **Netflix** – Handles millions of users by breaking services into smaller parts (recommendation engine, streaming service, user accounts).
- **Amazon** – Uses microservices for order management, inventory, shipping, payments.
- **Uber** – Independent services for ride matching, fare calculation, payments, notifications.
- **Healthcare** – Patient records, appointments, billing as separate microservices.
- **Banking** – Loan processing, customer onboarding, fraud detection as microservices.

Technology: Container - Kubernetes part 2



Advanced Container Concepts

1. Container Networking - Container networking enables containers to communicate with each other and with external applications.

- Every container gets its own network interface and IP.
- Docker supports Bridge, Host, and Overlay networks.
- Overlay networks connect containers running on different hosts.

Real-time example: In a food delivery app, the Order Service container communicates with the Payment Service container through a container network.

2. Volumes and Persistent Storage - Volumes provide persistent storage so data is not lost when a container stops or restarts.

- Container storage is temporary.
- Volumes are stored outside the container.
- The same volume can be reused by new containers.

Real-time example: Customer order history in a MySQL container remains available even after the container restarts.

3. Environment Variables and Secrets - Configuration values and sensitive information are supplied to containers without changing the application code.

- Environment variables store configuration such as port number or database URL.
- Secrets store passwords, API keys, and authentication tokens.
- Makes applications portable across development, testing, and production.

Real-time example: The Payment Service receives the Razorpay API key as a secret instead of storing it inside the Docker image.

4. Container Resource Management - Containers can be assigned CPU and memory limits to ensure efficient resource usage.

- CPU limits prevent one container from consuming all processor resources.
- Memory limits prevent excessive RAM usage.
- Helps multiple containers run efficiently on the same host.

Real-time example: During peak food orders, the Recommendation Service is limited to 1 GB RAM so it doesn't affect the Payment Service.

5. Container Image Optimization - Techniques used to build smaller, faster, and more secure container images.

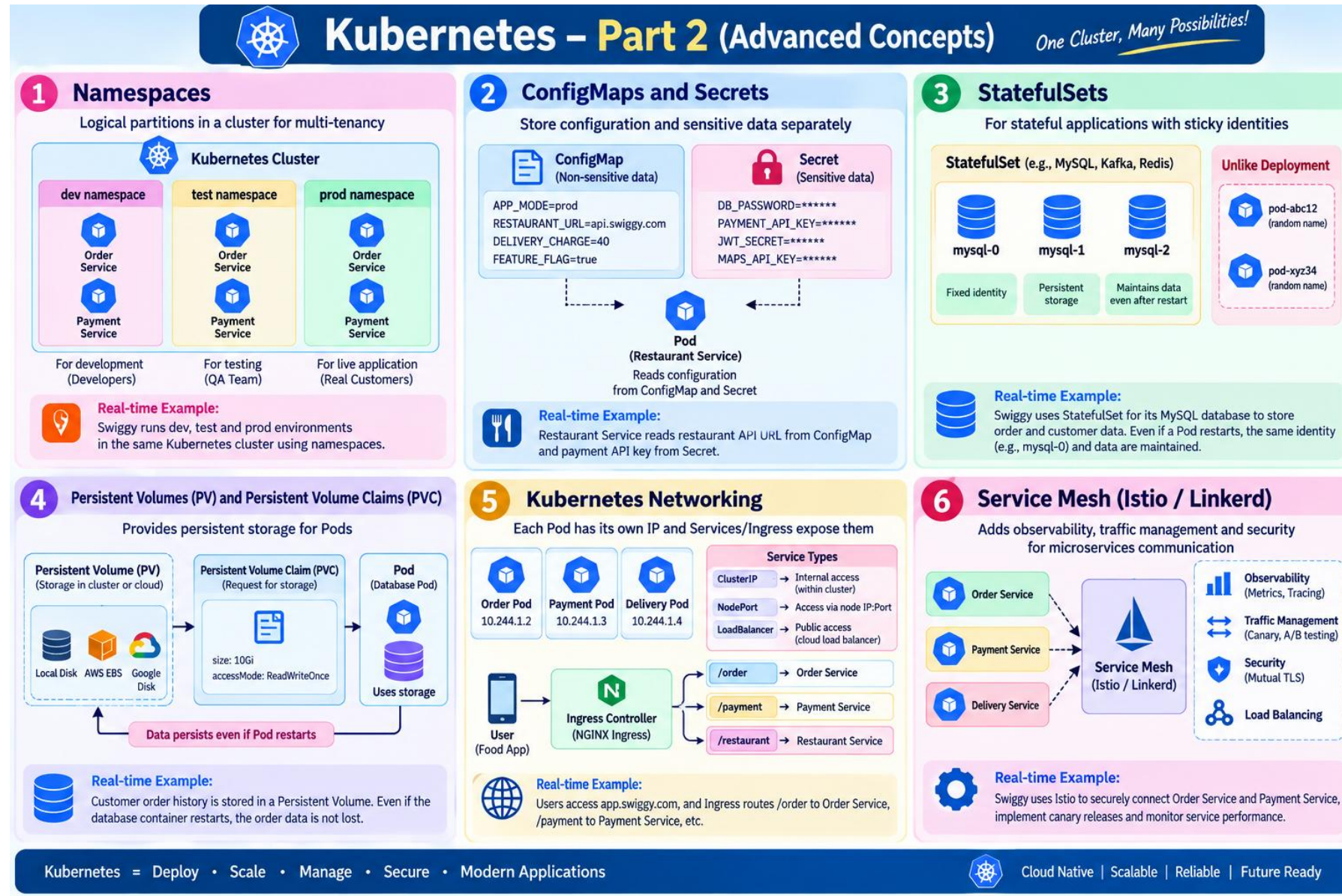
- Use lightweight base images (Alpine, Distroless).
- Use multi-stage builds to reduce image size.
- Remove unnecessary files and dependencies.

Real-time example: Swiggy reduces the Order Service image from 700 MB to 150 MB, making deployments much faster.

6. Container Registry - A repository where container images are stored and shared.

- Developers push images to a registry.
- Servers or Kubernetes pull images from the registry.
- Popular registries include Docker Hub, Amazon ECR, Azure ACR, and Google Artifact Registry.

Real-time example: After developers build a new Food Delivery app image, it is pushed to Docker Hub, and Kubernetes downloads the latest version for deployment.



Advanced Kubernetes Concepts

1) Namespaces: A Namespace is a logical partition inside a Kubernetes cluster. It allows multiple teams or environments to use the same Kubernetes cluster without interfering with each other.

Example: College ERP Example

- dev namespace → Developers test attendance application.
- test namespace → QA team tests the application.
- prod namespace → Students use the live application.

2) ConfigMaps and Secrets: These store configuration separately from the application.

ConfigMap - Stores non-sensitive configuration data.

Examples: Database hostname, Application port, Environment variables, API endpoint URL.

Secret - Stores sensitive information.

Examples: Database password, API keys, JWT tokens, Certificates.

3) StatefulSets - A StatefulSet manages applications that need persistent identity and storage.

Examples: MySQL, PostgreSQL, Cassandra, Kafka

4) Persistent Volumes (PV) and Persistent Volume Claims (PVC)

- Containers lose data when restarted.
- Kubernetes solves this using Persistent Storage.

Persistent Volume (PV)

- A storage resource available inside the cluster.
- It may come from: Local disk, AWS EBS, Azure Dis, Google Persistent Disk.

Persistent Volume Claim (PVC)

- A request made by a Pod asking Kubernetes for storage.

Flow

1. Storage exists (PV).
2. Pod requests storage (PVC).
3. Kubernetes attaches storage to Pod.

Example: MySQL Pod stores student records. Even if the Pod restarts, the database remains.

5) Kubernetes Networking

- Networking allows Pods and users to communicate.
- Each Pod gets its **own IP**.
- Services expose Pods using ClusterIP, NodePort, or LoadBalancer.
- **Ingress Controller** (e.g., NGINX Ingress) manages external access with routing rules.

6) Service Mesh (Istio/Linkerd)

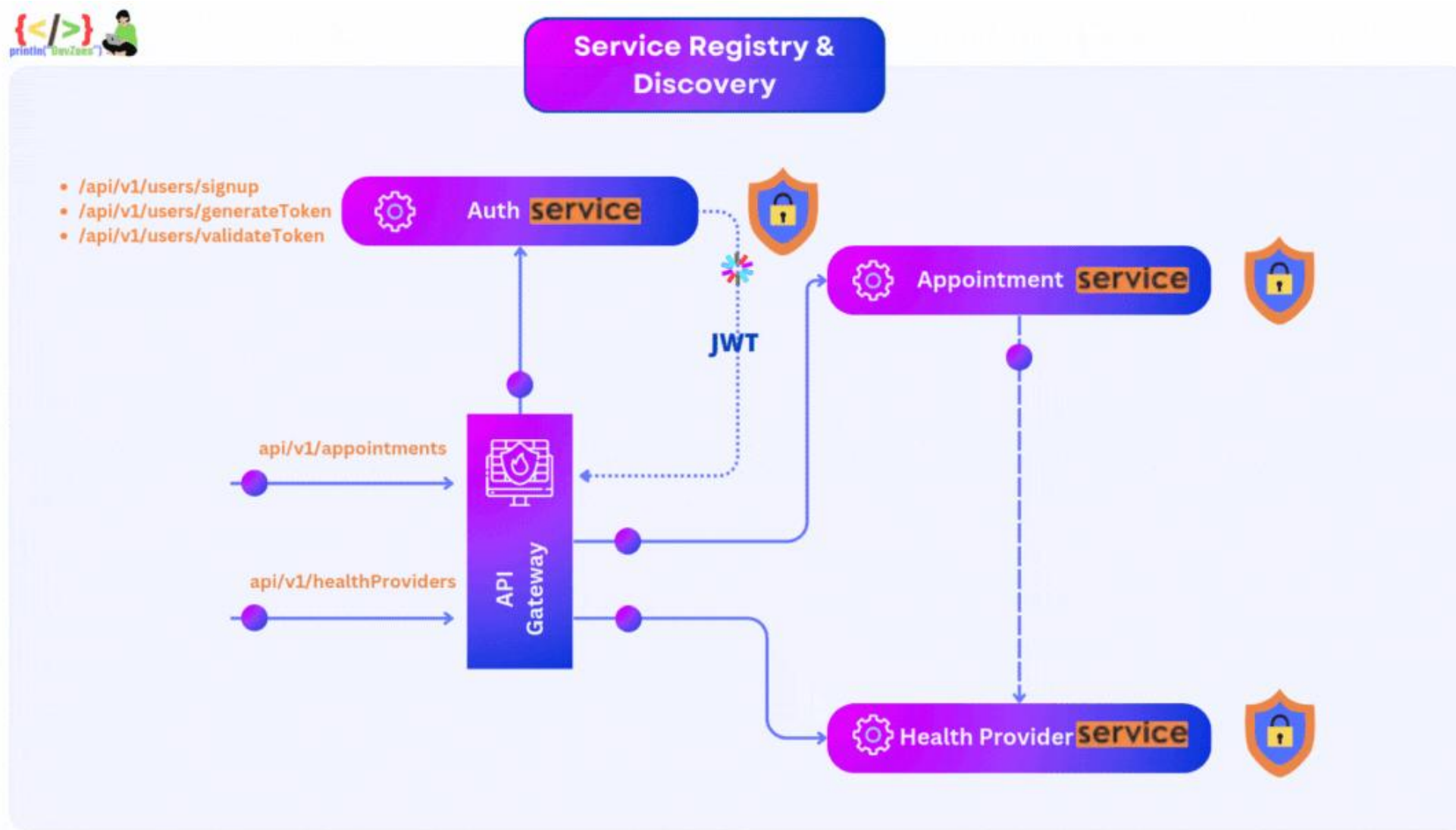
- A Service Mesh manages communication between microservices.
- Instead of adding networking code into applications, the mesh handles it.
- **Features:** Traffic routing, Monitoring, Security, Encryption, Load balancing
- **Real-Time Example:** Attendance Service communicates with Student Service securely using mutual TLS.
- Example: Canary releases, A/B testing, mutual TLS between services.

Technology: Inter-micro services communication

Introduction

- In a **microservices architecture**, multiple services work together to form a complete application.
 - Example: In an e-commerce app:
 - **User Service** (authentication)
 - **Order Service** (manages orders)
 - **Payment Service** (processes payments)
 - **Inventory Service** (stock updates)
 - For the system to function, services must **communicate** effectively with each other.
- 👉 This is called **Inter-Microservices Communication**.

Technology: Inter-micro services communication



Types of Inter-Service Communication

a) Synchronous Communication

- One service directly calls another and waits for a response.
- Protocols: **HTTP/REST, gRPC, GraphQL**.
- Example: Order Service calls Payment Service → waits for confirmation.

- ✓ **Pros:** Simple, widely used, direct response.
- ✗ **Cons:** Tight coupling, failure in one service can affect others.

b) Asynchronous Communication

- Services communicate via **message brokers** (no waiting).
- Protocols: **Message Queues, Event Streaming** (Kafka, RabbitMQ, AWS SQS).
- Example: Order Service places an "Order Created" event → Inventory Service consumes it.
- ✓ **Pros:** Decoupled, scalable, fault-tolerant.
- ✗ **Cons:** More complex, requires message management.

c) Event-Driven Communication

- Services publish events → other services **subscribe**.
- Example: Payment Success → triggers Shipping Service.
- Tools: **Apache Kafka, NATS, AWS SNS**.

Real-Time Applications

- **Netflix** → Uses gRPC + event streaming for video recommendations & personalization.
- **Uber** → gRPC + Kafka for real-time ride tracking, fare calculation.
- **Amazon** → REST APIs for order services + event-driven SQS for inventory.
- **Banking** → Asynchronous messaging ensures transaction reliability.
- **Healthcare** → Event-driven updates (e.g., patient record changes notify multiple systems).

Challenges in Inter-Service Communication

- **Latency** → Multiple calls can slow responses.
- **Fault Tolerance** → One service failure can cascade.
- **Data Consistency** → Especially in asynchronous models.
- **Security** → Must authenticate and encrypt communications.

Best Practices

- Use **API Gateway** to manage external/internal requests.
- Implement **circuit breakers** (e.g., Hystrix) for fault isolation.
- Use **service discovery** (e.g., Consul, Eureka) for locating services.
- Use **event-driven design** for scalability.
- Secure communication with **mTLS + JWT**.

Workshop for moving from monolithic to microservices components



Objective

- Build a small **monolithic Node.js + Express app** (single codebase).
- Break it into **two microservices** (User Service & Product Service).
- Show how services communicate (REST API calls).

Step 1: Monolithic Application

👉 A single Express app with **Users** and **Products** in one codebase.

// monolith-app.js

```
const express = require("express");
const app = express();
app.use(express.json());
```

// Users endpoint

```
app.get("/users", (req, res) => {
  res.json([ { id: 1, name: "Justina" } ]);
});
```

// Products endpoint

```
app.get("/products", (req, res) => {
  res.json([ { id: 101, name: "Laptop" } ]);
});
app.listen(3000, () => console.log("Monolith running on port 3000"));
```



Step 2: Break into Microservices

a) User Service

// user-service.js

```
const express = require("express");
```

```
const app = express();
```

```
app.get("/users", (req, res) => {
```

```
  res.json([ { id: 1, name: "Justina" }, { id: 2, name: "Michael" } ]);
```

```
});
```

```
app.listen(3001, () => console.log("User Service running on port 3001"));
```



b) Product Service

```
// product-service.js
```

```
const express = require("express");
```

```
const app = express();
```

```
app.get("/products", (req, res) => {
```

```
  res.json([ { id: 101, name: "Laptop" }, { id: 102, name: "Phone" } ]);
```

```
});
```

```
app.listen(3002, () => console.log("Product Service running on port 3002"));
```



Step 3: API Gateway (Optional, Advanced)

```
// api-gateway.js
```

```
const express = require("express");
```

```
const axios = require("axios");
```

```
const app = express();
```

```
app.get("/users", async (req, res) => {
```

```
  const response = await axios.get("http://localhost:3001/users");
```

```
  res.json(response.data);
```

```
});
```

```
app.get("/products", async (req, res) => {
```

```
  const response = await axios.get("http://localhost:3002/products");
```

```
  res.json(response.data);
```

```
});
```

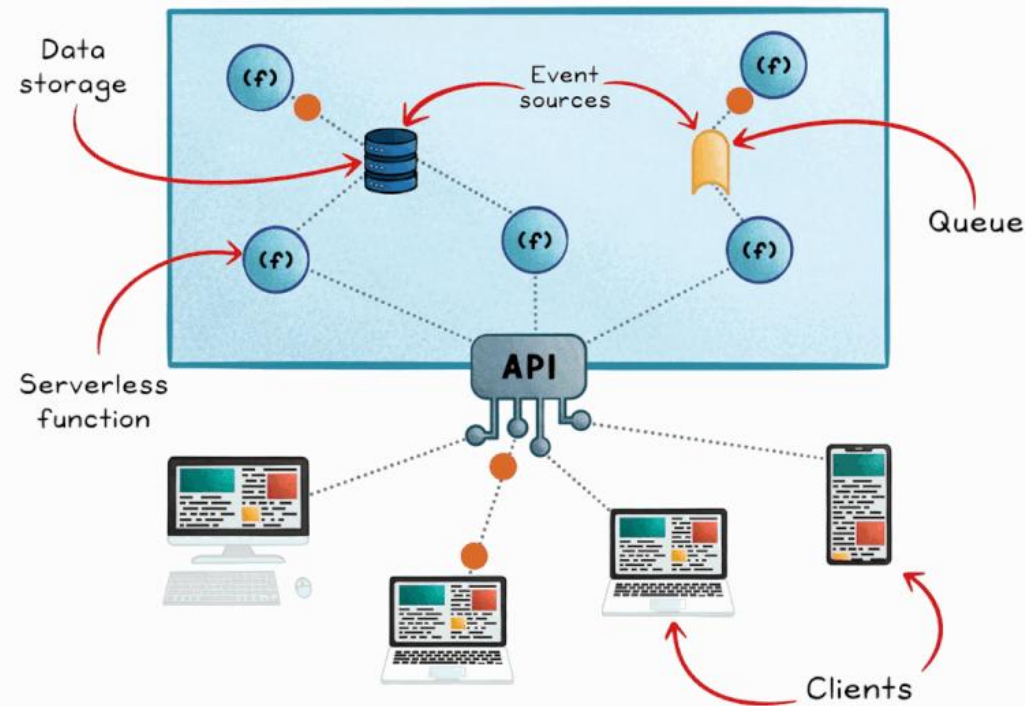
```
app.listen(3000, () => console.log("API Gateway running on port 3000"));
```

Serverless Architecture

Serverless Architecture

Serverless Architecture

by levelupcoding.com



Brought to you by



  @NikkiSiapno

  @LevelUpCoding

Serverless computing is a cloud-native development model that allows developers to build and run applications without managing servers.

Serverless Architecture is a cloud-native architecture in which the cloud provider automatically manages the infrastructure, while developers focus only on writing application logic.

- The cloud provider (AWS, Azure, GCP) automatically handles:
 - Server provisioning
 - Scaling
 - Maintenance
 - High availability
- Developers only **write code** as *functions*, and the cloud takes care of the infrastructure.
- Example
 - AWS Lambda
 - Azure Functions
 - Google Cloud Functions

Key Characteristics

- **No server management** → Servers exist, but abstracted away from developers.
- **Event-driven execution** → Functions trigger on events (HTTP request, file upload, DB change).
- **Pay-per-use** → Billing is based on execution time and requests (not idle server time).
- **Scalable by default** → Functions auto-scale based on workload.
- **Stateless functions** → Each invocation is independent, no persistence in memory.

Serverless vs Traditional Architecture

Feature	Traditional (Server-based)	Serverless
Server Management	Developer/DevOps manages	Cloud provider manages
Scaling	Manual / Auto-scaling setup	Auto, on-demand
Billing	Pay for uptime	Pay per request/execution
Use Case	Long-running apps	Event-driven workloads

Building Blocks of Serverless

- **FaaS (Function as a Service)**

- The heart of serverless. Developers write small, independent pieces of code (functions) that execute only when triggered by an event.
- Example: AWS Lambda, Azure Functions, Google Cloud Functions.
- Core idea: Write small independent functions → triggered by events.
- Example: A Lambda function generates an OTP whenever a customer logs in.

- **BaaS (Backend as a Service)**

- Ready-made cloud services that replace the need to build backend code for common tasks.
- Cloud-hosted services used directly instead of writing backend code.
- Example: Firebase Auth (authentication), AWS S3 (storage), DynamoDB (database).

- **Event Sources**

- Events are the **triggers** that make serverless functions run.
- API Gateway, message queues, file uploads, IoT devices.

Workflow - How They Fit Together

- **Event Source** triggers an action → e.g., user uploads an image.
- **FaaS function** runs → e.g., function resizes the image.
- **BaaS service** stores or processes the result → e.g., resized image is saved in S3.

So, **Serverless = Event Sources + Functions (FaaS) + Cloud Services (BaaS)**

In serverless, functions (FaaS) are triggered by events and often use backend services (BaaS) to get work done.

Advantages

- Faster development → focus only on code.
- Lower cost → pay only for execution.
- High scalability.
- Simplified operations (no patching, no server monitoring).

Challenges / Limitations

- **Cold start problem** → Delay in execution when function is idle for long.
- **Statelessness** → Functions can't store session data locally.
- **Vendor lock-in** → Hard to migrate between AWS Lambda, Azure Functions, etc.
- **Debugging complexity** → Harder compared to traditional apps.

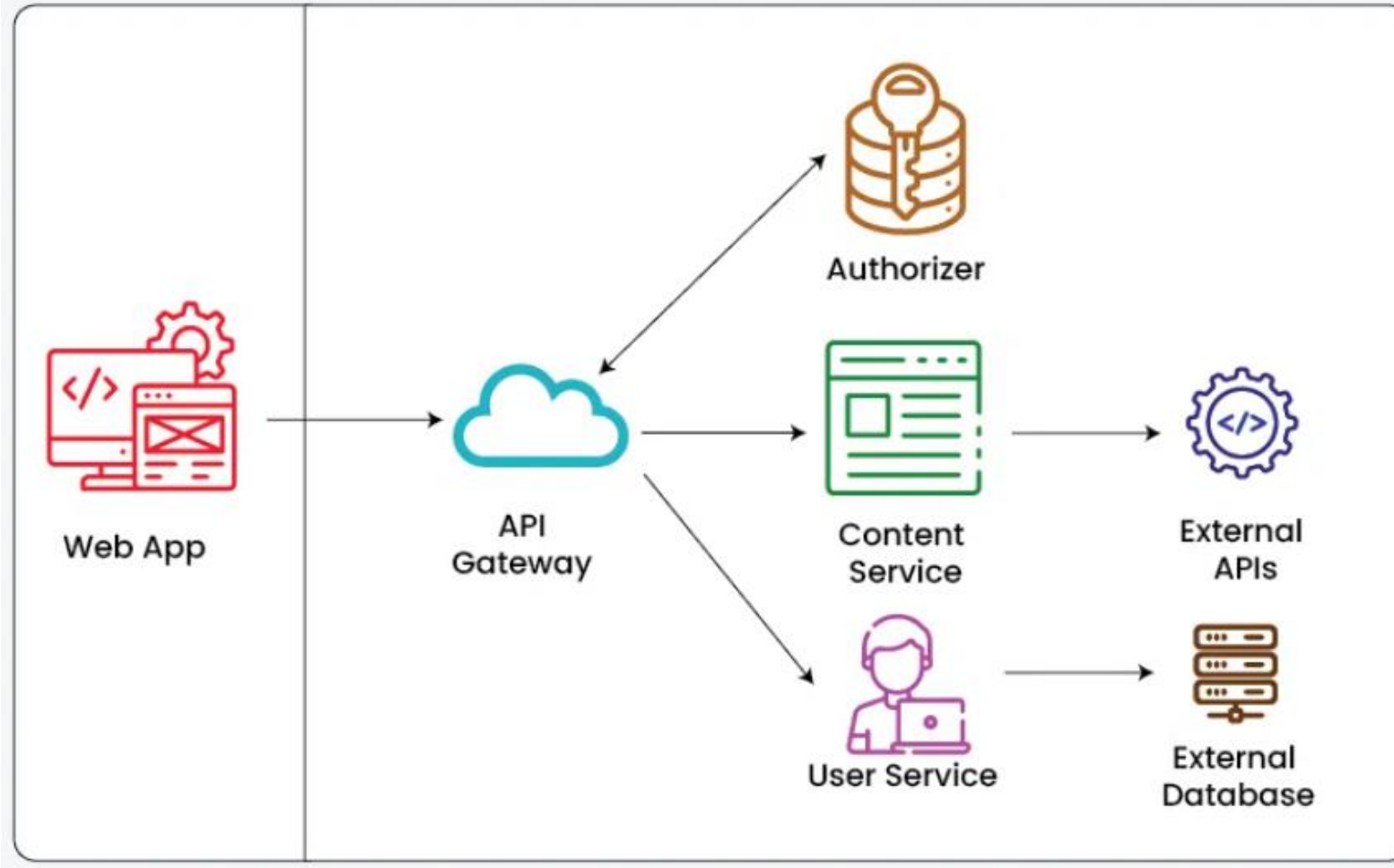
Real-Time Applications

- **Chatbots** → Serverless functions respond to user queries (Amazon Lex + Lambda).
- **IoT Processing** → Devices send data to the cloud → processed by Lambda.
- **Image/Video Processing** → Upload image → function resizes/filters it.
- **Event Notifications** → Trigger emails/SMS when events occur.
- **Data Pipelines** → Serverless functions clean and transform data before storage.

Introduction Concept: Function as a service

Function as a service (Faas)

What is Function as a Service (Faas)



Function as a service (Faas)

Introduction

Function as a Service (FaaS) is a **cloud computing model** where developers write **individual functions** that run in response to **events**.

- It is the **core of serverless computing**.
- Examples: **AWS Lambda, Azure Functions, Google Cloud Functions**.
- 👉 Instead of managing whole applications or servers, you just **deploy functions**.

Key Characteristics

- **Event-driven execution** → Functions triggered by events (API call, file upload, DB change).
- **Short-lived & stateless** → Functions run for a short time, no memory between runs.
- **Automatic scaling** → Scales up/down based on number of requests.
- **Pay-per-execution** → Only pay for the time function runs.
- **Cloud-managed** → Infrastructure, servers, and scaling are managed by provider.

Function as a service (FaaS)

How FaaS Works (Workflow)

1. Developer writes a function (e.g., processImage).
2. Function is uploaded to cloud provider (AWS, Azure, GCP).
3. An event occurs (e.g., image upload, API request).
4. Cloud provider runs the function automatically.
5. Function completes, resources are released.

Advantages

Cost-efficient → Pay only when functions run.

Faster development → Focus only on business logic.

High scalability → Handles 1 request or 1 million requests.

No infrastructure management.

Function as a service (FaaS)

Challenges

- **Cold start latency** → Delay if function hasn't run recently.
- **Stateless limitation** → No memory or session persistence.
- **Execution limits** → Functions usually have max execution time (e.g., AWS Lambda 15 mins).
- **Vendor lock-in** → Hard to move functions between AWS, Azure, GCP.

Real-Time Applications of FaaS

- **Image Processing** → Auto-resize or watermark images when uploaded to cloud storage.
- **Chatbots & Voice Assistants** → Functions triggered by user queries (e.g., Alexa Skills using AWS Lambda).
- **IoT Data Handling** → Sensors send data, functions process it in real time.
- **Notifications** → Auto-send SMS/Email when an event occurs.
- **Data Transformation** → Clean and format data before sending to databases.
- **API Backend** → Create APIs without managing servers (using API Gateway + Lambda).

Function as a service (FaaS)

Examples of FaaS Providers

- **AWS Lambda** → Most popular FaaS service.
- **Azure Functions** → Tight integration with Microsoft ecosystem.
- **Google Cloud Functions** → Lightweight function hosting.
- **IBM Cloud Functions** → Based on Apache OpenWhisk.

Example (AWS Lambda – Python)

```
def lambda_handler(event, context):  
    return {  
        'statusCode': 200,  
        'body': 'Hello from FaaS!'  
    }
```

Function as a service (FaaS)

Example: E-Commerce – Microservice and FAAS

- **Microservices** = long-running, independent, small services.
- **FaaS** = event-driven, short-lived, cloud-managed functions.

Microservices Example - E-commerce app

- Payment Microservice (handles payments)
- Inventory Microservice (manages stock)
- Order Microservice (process orders)

Runs continuously, communicates via APIs.

FaaS Example - E-commerce app for Payment microservice using Lambda

- Function: `validatePayment()` → runs only when a customer pays.
- Function: `sendEmail()` → runs only when order is confirmed.
- Function: `updateInventory()` → runs only when order placed.

Each is short-lived, event-driven, stateless.

Function as a service (Faas)

Question: In an Order Microservice that runs continuously and manages the full order lifecycle, identify which tasks/functions can be offloaded to a Function-as-a-Service (FaaS) model. Write down the possible FaaS functions.

Answer

Possible FaaS functions inside the **Order Microservice**:

- `ValidateCoupon()` → triggered when a customer applies a discount.
- `SendEmail()` → triggered after order confirmation.
- `GenerateInvoicePDF()` → triggered after payment is successful.
- *`SendSMSNotification()`*
- *`LogAnalyticsData()`*

Function as a service (FaaS)

Key Differences: Microservices vs FaaS

Aspect	Microservices	FaaS (Function-as-a-Service)
Definition	An application is broken into independent services (each service handles one business capability).	Application logic is broken into functions that run on demand in the cloud.
Deployment Unit	Each service is a containerized app (runs continuously).	Each function is a piece of code deployed to the cloud provider.
Execution Model	Runs 24/7 or until stopped. Always available.	Event-driven : Executes only when triggered (e.g., API call, file upload).
Scaling	Scales per service (e.g., auto-scale containers).	Scales per function (automatic, even to 1M requests).
State	Can be stateful or stateless depending on design.	Always stateless (no memory between runs, external DB needed).
Management	You manage services, APIs, communication, and sometimes scaling (with Kubernetes, Docker, etc.).	Cloud provider manages runtime, scaling, availability (you just write code).
Cost	Pay for compute resources (VMs/containers) whether used or idle.	Pay only when function runs (per execution, per ms).
Use Cases	Complex systems: e-commerce platforms, banking apps, enterprise SaaS.	Short tasks: image resize, data validation, notifications, IoT event processing.

Modelling: Functions

Introduction

- In **cloud-native applications**, complex systems are broken into **smaller, independent units**.
- **Functions** are the smallest deployable and executable unit of logic.
- Instead of building huge monolithic applications, developers **model business logic as functions**.
- 👉 This approach is **event-driven, stateless, and cloud-managed**.

Why Model as Functions?

- **Simplicity** → Focus only on “what function does,” not on infrastructure.
- **Reusability** → Same function can be used across services.
- **Scalability** → Functions scale independently.
- **Maintainability** → Easier debugging and updates.
- **Cost-efficiency** → Pay only when functions execute.

Key Characteristics of Functions

- **Stateless** → Each execution is independent, no memory of past runs.
- **Event-driven** → Triggered by external events (API call, file upload, DB update).
- **Single responsibility** → Each function does one thing well.
- **Composable** → Functions can be chained together to build workflows.
- **Cloud-Managed lifecycle** → Deployment, scaling, and monitoring handled by provider.

Modeling Functions in Cloud-Native Systems

When designing a cloud-native app:

- **Identify Events** → (user uploads file, IoT device sends data).
- **Define Functions** → (resize image, validate data, send notification).
- **Choose Triggers** → (HTTP request, database change, message queue).
- **Connect Functions** → Combine multiple functions to create workflows (using event buses or orchestrators).
- **Deploy Independently** → Each function can evolve separately.

Real-Time Applications

- **E-commerce** →
 - Function: “Process payment”
 - Function: “Send order confirmation email”
 - Function: “Update inventory”
- **Social Media** →
 - Function: “Resize image when uploaded”
 - Function: “Detect inappropriate content”
 - Function: “Send push notifications”
- **IoT Applications** →
 - Function: “Collect sensor data”
 - Function: “Trigger alert if threshold crossed”
 - Function: “Log data into database”
- **Healthcare** →
 - Function: “Analyze uploaded medical image”
 - Function: “Notify doctor if anomaly found”
- **Education Platforms** →
 - Function: “Auto-grade quiz submissions”
 - Function: “Send feedback to students”

Example (Node.js Function)

Benefits of Function Modeling

- Faster innovation → Focus on logic, not infrastructure.
- Easy to test small pieces.
- Fault isolation → Failure in one function won't crash entire system.
- Encourages microservices mindset.

Challenges

- **Cold start delays** → May take time if function not recently used.
- **Stateless only** → Persistent data must be stored externally.
- **Complex orchestration** → Multiple functions require coordination.
- **Vendor lock-in** → Hard to migrate functions between providers.

Technology: Elements of functions and workflow implementation mechanism

Technology: Elements of functions and workflow implementation mechanism

Introduction

- **Functions** are the building blocks in **serverless / Function-as-a-Service (FaaS)** environments.
- Real-world apps usually need **multiple functions** that must work together in a **workflow**.
- Cloud providers give **technologies and tools** to implement, connect, and manage these workflows.

Elements of Functions (Basic Components)

- **Trigger / Event Source**
 - Defines **what starts the function**.
 - Examples: HTTP request, file upload, message in queue, IoT event.
- **Input Data / Event Payload**
 - The **data passed** to the function when triggered.
 - Example: JSON object of a new order.

Technology: Elements of functions and workflow implementation mechanism

- **Execution Environment**
 - The **runtime** where the function executes (Node.js, Python, Java, Go, etc.).
 - Managed by cloud provider.
- **Function Logic**
 - The **core code** that performs the action.
 - Example: resizing an image, validating payment.
- **Output / Response**
 - What the function returns after execution.
 - Example: “Order confirmed”, “Payment success”.
- **State Management (Optional)**
 - Since functions are stateless, external services (DB, cache, storage) store persistent state.
- **Scaling & Monitoring (Handled by Cloud Provider)**
 - Auto-scaling, logging, error tracking.

Technology: Elements of functions and workflow implementation mechanism

Workflow Implementation Mechanism: A **workflow** = connecting multiple functions together to achieve a larger task.

Steps in Workflow Implementation

1. Define Business Process → Break into small tasks (functions).

Example: “*Order Processing*” can be broken into:

- Validate payment
- Update inventory
- Send confirmation email

2. Sequence / Parallel Execution → Functions may run one after another or simultaneously.

Example:

- *Sequential*: Validate payment → then update inventory.
- *Parallel*: After payment, send email **and** generate invoice at the same time.

Technology: Elements of functions and workflow implementation mechanism

3. Event Chaining → Output of one function becomes input for another.

Example:

- Payment function → outputs “PaymentSuccess = true”.
- That output triggers the next function: “Send Email”.

4. Orchestration vs. Choreography:

- **Orchestration:** Central controller decides flow (e.g., AWS Step Functions, Azure Logic Apps).

Example: Validate Payment, Update Inventory, Send Email

- **Choreography:** Functions react to events without central control (event-driven, pub/sub).\ul>- Example: PaymentSuccess event is published, Email service *listens* and sends email, Inventory service *listens* and updates stock.

Technology: Elements of functions and workflow implementation mechanism

Real-Time Applications of Functions + Workflows

- **E-Commerce Order Processing**

- Function 1: Validate payment
- Function 2: Update inventory
- Function 3: Send confirmation email
- Orchestrator: Ensures sequence is followed.

- **Healthcare Workflow**

- Function 1: Upload medical image → store in cloud.
- Function 2: Analyze image with AI.
- Function 3: Notify doctor if anomaly detected.

- **IoT Smart Home**

- Function 1: Receive temperature reading.
- Function 2: Trigger AC if above threshold.
- Function 3: Log data to storage.

Technology: Elements of functions and workflow implementation mechanism

Education

- Function 1: Auto-grade quiz submission.
- Function 2: Generate result PDF.
- Function 3: Email result to student.

Example Workflow (Pseudocode-like)

OrderWorkflow:

Step1: ValidatePaymentFunction

Step2: UpdateInventoryFunction

Step3: SendEmailFunction

👉 Each function runs independently, connected by workflow orchestration.

Technology: Elements of functions and workflow implementation mechanism

Benefits of Functions + Workflow

- **Scalability** → Each step scales independently.
- **Modularity** → Easy to add/remove functions.
- **Resilience** → If one function fails, retry only that part.
- **Faster Development** → Teams can build and deploy independently.

Challenges

- **Debugging complexity** across multiple functions.
- **Latency** if too many chained functions.
- **Vendor lock-in** with orchestration tools.
- **Cost management** if workflows run very frequently.

Thank You